ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

Departamento de Ingeniería

Trabajo Práctico 01

Ejercicio 1: Permutaciones con Restricciones (Fuerza Bruta)

Asignatura: Análisis y Diseño de Algoritmos
Profesor: Carlos Alberto Álvarez Henao

Andrés Felipe Vélez Álvarez
Sebastián Salazar Henao
Nathalia Valentina Cardoza Azuaje

5 de abril de 2026
Universidad EAFIT

Índice

1. Descripción Teórica del Problema **3**

2. Enfoques de Solución **3**
2.1. Fuerza Bruta . . . 3
2.2. Por qué este enfoque es Fuerza Bruta y no Backtracking . . . 3
2.3. Justificación del uso de `next_permutation` de la STL . . . 4
2.4. Backtracking (referencia comparativa) . . . 5
2.5. Comparación de Enfoques . . . 5

3. Pseudocódigo **5**
3.1. Fuerza Bruta . . . 5
3.2. Algoritmo interno de `next_permutation` . . . 6

4. Implementación en C++ **6**

5. Ejemplos de Entrada y Salida **9**
5.1. Ejemplo 1 — Conjunto {1, 3, 4} (n=3) . . . 9
5.2. Ejemplo 2 — Conjunto {1, 2, 3} (n=3) . . . 10
5.3. Ejemplo 3 — Conjunto {1, 2, 3, 4} (n=4) . . . 10
5.4. Ejemplo 4 — Conjunto {1, 5, 20} (n=3) — Elementos alejados . . . 11

6. Comparación de Tiempos de Ejecución **11**

7. Análisis de Complejidad **12**
7.1. Complejidad Temporal . . . 12
7.2. Complejidad Espacial . . . 12

8. Respuestas a las Preguntas Guía **12**
8.1. ¿Qué ocurre con la proporción de permutaciones válidas cuando crece n? . . . 12
8.2. ¿Dónde se podría aplicar poda con Backtracking? . . . 13
8.3. ¿Por qué `next_permutation` genera exactamente $n!$ permutaciones? . . . 13

9. Conclusiones **14**

1. Descripción Teórica del Problema

Dado un conjunto de n enteros positivos distintos $A = \{a_0, a_1, \ldots, a_{n-1}\}$, se desea generar todas las permutaciones posibles del conjunto y determinar cuáles satisfacen la siguiente restricción de adyacencia:

$$P[i] \leq 2 \cdot P[i+1] \quad \forall\, i \in \{0, 1, \ldots, n-2\} \tag{1}$$

Es decir, en toda permutación válida ningún elemento puede ser mayor que el doble del elemento que le sigue. El programa debe: (a) generar todas las permutaciones del conjunto mediante fuerza bruta (enumeración exhaustiva del factorial de permutaciones), (b) filtrar las que cumplen la restricción, y (c) reportar el total de permutaciones generadas, el total de válidas y el listado de éstas.

El número total de permutaciones de n elementos distintos es $n!$, lo que determina directamente la complejidad del enfoque. A modo de referencia:

n	$n!$	Tiempo estimado
5	120	Instantáneo
8	40 320	Muy rápido
10	3 628 800	Rápido
12	479 001 600	Lento
13	6 227 020 800	Impracticable

2. Enfoques de Solución

2.1. Fuerza Bruta

La fuerza bruta aborda el problema mediante enumeración exhaustiva: se generan absolutamente todas las $n!$ permutaciones del conjunto y, una vez construida cada una, se verifica si cumple la restricción $P[i] \leq 2 \cdot P[i+1]$ para todos sus pares consecutivos. Las permutaciones que pasan el filtro se cuentan e imprimen; las que no, se descartan.

Proceso paso a paso:

1. Se lee el conjunto de n enteros y se ordena de forma ascendente. Esto garantiza comenzar desde la permutación lexicográficamente menor, condición necesaria para que `next_permutation` recorra exactamente las $n!$ permutaciones.

2. Se itera sobre todas las permutaciones mediante un bucle `do-while` combinado con la función `next_permutation`.

3. En cada iteración se verifica la restricción recorriendo los $n-1$ pares consecutivos. Si algún par la viola, la permutación se descarta con `break`.

4. Al terminar, se reportan los totales y el listado de permutaciones válidas.

2.2. Por qué este enfoque es Fuerza Bruta y no Backtracking

Aunque ambos algoritmos resuelven el mismo problema y producen los mismos resultados, la diferencia fundamental radica en *cuándo* se evalúa la restricción durante la construcción de cada permutación.

En la fuerza bruta implementada, la permutación se construye **completamente** antes de verificar si es válida. Es decir, se genera la permutación entera de n elementos y sólo al final se recorren sus pares consecutivos para aceptarla o descartarla. En ningún momento se interrumpe la construcción anticipadamente: si los primeros dos elementos ya violan la restricción, el algoritmo termina de armar la permutación completa antes de detectarlo y descartarla.

En el *backtracking*, en cambio, la verificación ocurre **durante** la construcción, posición por posición. Al colocar el elemento en la posición i, se comprueba inmediatamente si viola la restricción con el elemento en $i-1$. Si la viola, se poda la rama completa: no se exploran ninguna de las $(n-i-1)!$ sub-permutaciones que se derivan de ese prefijo inválido.

La clave definitoria es la **ausencia de poda**:

- **Fuerza Bruta:** genera las $n!$ permutaciones sin excepción, evalúa la restricción *a posteriori* sobre la permutación completa.
- **Backtracking:** construye parcialmente y poda *a priori* las ramas que no pueden llevar a soluciones válidas.

Por tanto, aunque el código utilice `next_permutation` (que internamente aplica un algoritmo de generación lexicográfica eficiente), la estrategia de búsqueda sigue siendo fuerza bruta: el espacio de búsqueda explorado es siempre el total de las $n!$ permutaciones, sin ninguna reducción anticipada basada en la restricción del problema.

2.3. Justificación del uso de `next_permutation` de la STL

Para generar las permutaciones se utiliza la función `std::next_permutation` de la biblioteca estándar de C++ (`<algorithm>`) en lugar de implementar un algoritmo propio. Esta decisión se justifica por las siguientes razones:

1. **Corrección garantizada.** La implementación de la STL ha sido ampliamente probada y verificada. Una implementación propia del algoritmo de generación lexicográfica puede introducir errores sutiles, por ejemplo en el manejo de los índices de frontera o en la inversión del sufijo.
2. **Completitud del recorrido.** `next_permutation` garantiza recorrer exactamente las $n!$ permutaciones en orden lexicográfico estricto, sin repetir ni omitir ninguna, siempre que el arreglo de entrada esté ordenado ascendentemente al inicio del bucle.
3. **Eficiencia.** El algoritmo interno opera en tiempo amortizado $O(n)$ por llamada, que es el óptimo posible para generar la siguiente permutación en orden lexicográfico.
4. **Legibilidad.** El patrón `do-while` con `next_permutation` expresa de forma directa la intención del algoritmo sin oscurecer la lógica principal con detalles de implementación del generador.
5. **Portabilidad.** Al ser parte del estándar ISO de C++, la función está disponible en cualquier compilador compatible sin dependencias externas.

La implementación propia `next_permutation_test` se incluye en el código exclusivamente con fines educativos, para evidenciar el mecanismo interno del algoritmo de generación lexicográfica.

2.4. Backtracking (referencia comparativa)

El backtracking mejoraría el enfoque construyendo la permutación posición por posición y verificando la restricción en cada paso antes de agregar el siguiente elemento. Si en la posición i la condición $A[i-1] \leq 2 \cdot A[i]$ no se cumple, se poda esa rama completa sin explorar ninguna de sus sub-permutaciones. A esta eliminación anticipada se le llama **poda**.

2.5. Comparación de Enfoques

Aspecto	**Fuerza Bruta**	**Backtracking**
Espacio de búsqueda	Todas las $n!$ permutaciones	Árbol podado
Poda	No	Sí (viola restricción $\Rightarrow$ descarta)
Completitud	Sí	Sí
Corrección	Sí	Sí
Complejidad peor caso	$O(n! \cdot n)$	$\leq O(n! \cdot n)$
Eficiencia práctica	Inferior para n grande	Mejor con restricciones fuertes

3. Pseudocódigo

3.1. Fuerza Bruta

```
leer n
leer A[0..n-1]   // conjunto de enteros distintos
ordenar A asc    // garantiza comenzar desde la permutación menor

total_generadas = 0
total_validas   = 0

// Calcular n! para verificación
cont = 1
para i en [1..n]:
    cont = cont * i

// Bucle principal: recorre todas las n! permutaciones
hacer:
    total_generadas += 1
    valida = verdadero

    // Verificar la restricción en cada par consecutivo
    para i en [0..n-2]:
        si A[i] > 2 * A[i+1]:
            valida = falso
            break   // no hace falta seguir verificando

    si valida:
        imprimir A
        total_validas += 1
```

```

mientras next_permutation(A) exista

imprimir "Total permutaciones posibles: ",  cont
imprimir "Total permutaciones generadas:", total_generadas
imprimir "Total permutaciones válidas: ",  total_validas
```

Listing 1: Pseudocódigo: Permutaciones con Restricción por Fuerza Bruta

La función `next_permutation` recibe el arreglo y lo reorganiza en su siguiente permutación en orden lexicográfico, modificándolo directamente en memoria. Retorna `true` si existe una permutación siguiente, o `false` cuando ya se alcanzó la permutación máxima (orden descendente).

3.2. Algoritmo interno de `next_permutation`

```
next_permutation(inicio, fin):
    n = fin - inicio
    si n <= 1: retornar falso

    // Paso 1: buscar punto de quiebre de derecha a izquierda
    i = n - 2
    mientras i >= 0 y inicio[i] >= inicio[i+1]:
        i = i - 1

    si i == -1: retornar falso   // última permutación (todo
       descendente)

    // Paso 2: buscar el primer elemento mayor que inicio[i] desde
       la derecha
    j = n - 1
    mientras inicio[j] <= inicio[i]:
        j = j - 1

    // Paso 3: intercambiar inicio[i] e inicio[j]
    swap(inicio[i], inicio[j])

    // Paso 4: invertir el sufijo desde i+1 hasta el final
    invertir(inicio[i+1..n-1])

    retornar verdadero
```

Listing 2: Pseudocódigo interno: next_permutation

4. Implementación en C++

La implementación completa se encuentra en el archivo `main_ejercicio1.cpp` adjunto al presente informe. El código se organiza en los siguientes bloques:

- **`main`:** lectura del conjunto, ordenamiento mediante burbuja, bucle principal con `do-while`, medición de tiempo y reporte de resultados.

- `factorial`: calcula $n!$ de forma iterativa para reportar el total teórico y verificar que `total_generadas == factorial(n)`.
- `next_permutation_test`: implementación propia del algoritmo de generación lexicográfica, incluida para evidenciar el mecanismo interno.
- **Medición de tiempo:** se usa `chrono::high_resolution_clock` para medir únicamente el tiempo del bucle principal.

Para compilar y ejecutar:

```
g++ -O2 -std=gnu++17 -o solucion permutaciones.cpp
./solucion
```

```cpp
#include <iostream>
#include <algorithm>
#include <chrono>
using namespace std;

int  factorial(int n);
bool next_permutation_test(int* inicio, int* fin);

int main() {

    int n = 0, total_generadas = 0, total_validas = 0, aux = 0;
    bool valida;

    cout << "Ingrese el tamaño del arreglo: ";
    cin >> n;
    int A[n];

    for (int i = 0; i < n; i++) {
        cout << "Ingrese el elemento " << i + 1 << ": ";
        cin >> A[i];
    }

    // Ordenar ascendente mediante burbuja
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (A[j] > A[j+1]) {
                aux   = A[j];
                A[j]  = A[j+1];
                A[j+1] = aux;
            }
        }
    }

    // Inicio del cronómetro: mide solo el bucle principal
    auto inicio = chrono::high_resolution_clock::now();

    do {
        total_generadas++;
```

```
        valida = true;

        for (int i = 0; i < n - 1; i++) {
            if (A[i] > 2 * A[i+1]) {
                valida = false;
                break;
            }
        }

        if (valida) {
            cout << "Permutación válida: ";
            for (int i = 0; i < n; i++)
                cout << A[i] << " ";
            cout << endl;
            total_validas++;
        }

    } while (next_permutation(A, A + n));

    auto fin = chrono::high_resolution_clock::now();
    chrono::duration<double> duracion = fin - inicio;

    cout << "Total permutaciones posibles:  " << factorial(n)
              << endl;
    cout << "Total permutaciones generadas: " << total_generadas
            << endl;
    cout << "Total permutaciones válidas:   " << total_validas
              << endl;
    cout << "Tiempo de ejecución: "           << duracion.count()
         << " segundos" << endl;

    return 0;
}

// Calcula n! de forma iterativa
int factorial(int n) {
    int cont = 1;
    for (int i = 1; i < n; i++)
        cont = cont * (i + 1);
    return cont;
}

// Implementación propia de next_permutation (fines educativos)
bool next_permutation_test(int* inicio, int* fin) {
    int n = fin - inicio;
    if (n <= 1) return false;

    // Paso 1: buscar punto de quiebre de derecha a izquierda
    int i = n - 2;
    while (i >= 0 && inicio[i] >= inicio[i + 1]) i--;
    if (i == -1) return false;
```

```

    // Paso 2: buscar el sucesor de inicio[i]
    int j = n - 1;
    while (inicio[j] <= inicio[i]) j--;

    // Paso 3: intercambiar
    int temp  = inicio[i];
    inicio[i] = inicio[j];
    inicio[j] = temp;

    // Paso 4: invertir el sufijo desde i+1
    int izq = i + 1, der = n - 1;
    while (izq < der) {
        temp        = inicio[izq];
        inicio[izq] = inicio[der];
        inicio[der] = temp;
        izq++; der--;
    }
    return true;
}
```

Listing 3: Implementación completa: permutaciones.cpp

5. Ejemplos de Entrada y Salida

Los siguientes ejemplos corresponden a ejecuciones reales del programa. Los tiempos se midieron con `chrono::high_resolution_clock`.

5.1. Ejemplo 1 — Conjunto {1, 3, 4} (n=3)

El conjunto del enunciado. Con tres elementos existen $3! = 6$ permutaciones. El valor 4 supera el doble de 1, lo que descarta varias combinaciones.

Salida real del programa:

```
=============================================
Permutación válida: 1 3 4
Permutación válida: 1 4 3
=============================================
Total permutaciones posibles:  6
Total permutaciones generadas: 6
Total permutaciones válidas:   2
Tiempo de ejecución: 0.0009147 segundos
```

Verificación manual:

Permutación	Par (0,1)	Par (1,2)	Válida	Razón
$[1,3,4]$	$1 \leq 6$ ✓	$3 \leq 8$ ✓	Sí	Ambos pares cumplen
$[1,4,3]$	$1 \leq 8$ ✓	$4 \leq 6$ ✓	Sí	Ambos pares cumplen
$[3,1,4]$	$3 \leq 2$ ×	—	No	$3 > 2 \cdot 1$
$[3,4,1]$	$3 \leq 8$ ✓	$4 \leq 2$ ×	No	$4 > 2 \cdot 1$
$[4,1,3]$	$4 \leq 2$ ×	—	No	$4 > 2 \cdot 1$
$[4,3,1]$	$4 \leq 6$ ✓	$3 \leq 2$ ×	No	$3 > 2 \cdot 1$

5.2. Ejemplo 2 — Conjunto {1, 2, 3} (n=3)

Con elementos consecutivos la restricción es menos estricta. Nótese que $[3,2,1]$ es válida porque $3 \leq 2 \cdot 2 = 4$ y $2 \leq 2 \cdot 1 = 2$: la igualdad cumple la condición $\leq$.

```
==============================================
Permutación válida: 1 2 3
Permutación válida: 1 3 2
Permutación válida: 2 1 3
Permutación válida: 3 2 1
==============================================
Total permutaciones posibles:  6
Total permutaciones generadas: 6
Total permutaciones válidas:   4
Tiempo de ejecución: 0.0014084 segundos
```

5.3. Ejemplo 3 — Conjunto {1, 2, 3, 4} (n=4)

Con $n = 4$ el espacio de búsqueda crece a $4! = 24$ permutaciones y la proporción de válidas baja al 33 %.

```
==============================================
Permutación válida: 1 2 3 4
Permutación válida: 1 2 4 3
Permutación válida: 1 3 2 4
Permutación válida: 1 3 4 2
Permutación válida: 1 4 2 3
Permutación válida: 1 4 3 2
Permutación válida: 2 1 3 4
Permutación válida: 2 1 4 3
Permutación válida: 3 2 1 4
Permutación válida: 3 4 2 1
Permutación válida: 4 2 1 3
Permutación válida: 4 3 2 1
==============================================
Total permutaciones posibles:  24
Total permutaciones generadas: 24
Total permutaciones válidas:   12
Tiempo de ejecución: 0.0042051 segundos
```

5.4. Ejemplo 4 — Conjunto {1, 5, 20} (n=3) — Elementos alejados

Este caso ilustra cómo la distancia entre los valores del conjunto afecta el número de válidas. Con elementos muy separados la restricción es muy estricta.

```
==============================================
Permutación válida: 1 5 20
==============================================
Total permutaciones posibles:  6
Total permutaciones generadas: 6
Total permutaciones válidas:   1
Tiempo de ejecución: 0.00048 segundos
```

Comparando con $\{1, 2, 3\}$ que produjo 4 válidas, aquí solo se obtiene 1, confirmando que la proporción de válidas depende tanto de n como de los valores específicos del conjunto.

6. Comparación de Tiempos de Ejecución

Se ejecutó el programa con el conjunto $\{1, 2, \ldots, n\}$ para distintos valores de n. Los tiempos se obtuvieron con `chrono::high_resolution_clock` midiendo únicamente el bucle principal.

n	$n!$	Válidas	Tiempo (s)	Factor
6	720	144	0.0752	—
8	40 320	2 880	2.0437	×27x
10	3 628 800	86 400	82.0305	×40x
12	479 001 600	3 628 800	3 962.63*	×40x

* Valor estimado por extrapolación a partir de los tiempos medidos para $n = 6$, $n = 8$ y $n = 10$. La ejecución real de $n = 12$ supera los 45 minutos y no fue completada, lo cual es consistente con la complejidad $O(n! \cdot n)$ del algoritmo.

Análisis de los resultados:

- **Ciclo** $n = 6$**:** 720 permutaciones generadas en 0,075 segundos. El programa responde de forma instantánea. Se obtuvieron 144 permutaciones válidas (20 %).
- **Ciclo** $n = 8$**:** 40 320 permutaciones en 2,04 segundos. Tiempo perceptible pero práctico. Se obtuvieron 2 880 válidas (7,1 %).
- **Ciclo** $n = 10$**:** 3 628 800 permutaciones en 82 segundos. El tiempo ya es impracticable para uso interactivo. Se obtuvieron 86 400 válidas (2,4 %).
- **Ciclo** $n = 12$**:** con 479 millones de permutaciones el tiempo estimado supera los 45 minutos, confirmando que $n = 12$ es el límite práctico del algoritmo.

Se observa que el tiempo crece aproximadamente ×40 cada dos niveles, lo que es consistente con el factor $12 \times 11 = 132$ de crecimiento de 10! a 12!.

7. Análisis de Complejidad

7.1. Complejidad Temporal

La complejidad temporal del algoritmo se descompone en dos partes:

- El bucle principal itera exactamente $n!$ veces, una por cada permutación posible.
- Dentro de cada iteración, el bucle de validación recorre como máximo $n-1$ pares consecutivos: $O(n)$.

Por lo tanto la complejidad temporal total es:

$$T_{FB}(n) = O(n! \cdot n) \tag{2}$$

Este es siempre el mismo costo sin importar el conjunto de entrada, ya que la fuerza bruta siempre recorre las $n!$ permutaciones completas antes de descartar cualquiera.

n	$n!$	$n! \cdot n$	Operaciones aprox.
8	40 320	322 560	$3{,}2 \times 10^5$
10	3 628 800	36 288 000	$3{,}6 \times 10^7$
12	479 001 600	5 748 019 200	$5{,}7 \times 10^9$

7.2. Complejidad Espacial

Estructura	Espacio
Arreglo A de n enteros	$O(n)$
Variables auxiliares (contadores, aux, valida)	$O(1)$
Permutaciones intermedias (no se almacenan)	$O(1)$
Total	$O(n)$

El algoritmo tiene complejidad espacial $O(n)$ porque en ningún momento almacena todas las permutaciones en memoria: procesa cada una en el lugar y la descarta antes de generar la siguiente.

8. Respuestas a las Preguntas Guía

8.1. ¿Qué ocurre con la proporción de permutaciones válidas cuando crece n?

La proporción de permutaciones válidas **disminuye drásticamente** a medida que n crece. Esto ocurre porque la restricción $P[i] \leq 2 \cdot P[i+1]$ debe cumplirse de forma simultánea en todos los $n-1$ pares consecutivos. Cuantos más pares haya que satisfacer al mismo tiempo, menor es la probabilidad de que todos lo hagan.

Es importante notar además que el número exacto de válidas depende de los valores específicos del conjunto, no sólo de n: conjuntos con elementos muy separados entre sí generan menos válidas que conjuntos con elementos cercanos.

Usando los datos reales obtenidos con el conjunto $\{1, 2, \ldots, n\}$:

n	Total ($n!$)	Válidas (medidas)	Proporción	Observación
6	720	144	20,0 %	Medido experimentalmente
8	40 320	2 880	7,1 %	Medido experimentalmente
10	3 628 800	86 400	2,4 %	Medido experimentalmente
12	479 001 600	3 628 800	0,5 %	Medido experimentalmente

Se observa que la proporción disminuye de 20 % a 2,4 % al pasar de $n = 6$ a $n = 10$, una reducción de casi un orden de magnitud en sólo 4 niveles. Mientras el total crece factorialmente ($\times 5040$ de $n = 6$ a $n = 10$), las válidas crecen mucho más lentamente ($\times 600$), haciendo que la proporción tienda a cero conforme n aumenta. Esto evidencia la ineficiencia creciente de la fuerza bruta.

8.2. ¿Dónde se podría aplicar poda con Backtracking?

Con backtracking se construiría la permutación posición por posición, verificando la restricción en cada paso antes de agregar el siguiente elemento. La poda ocurriría de la siguiente manera:

- En la posición 0 se prueba cada elemento disponible del conjunto.
- Antes de colocar un elemento en la posición i, se verifica si el elemento ya colocado en la posición $i - 1$ cumple $A[i-1] \leq 2 \cdot A[i]$.
- Si no cumple $\Rightarrow$ se poda toda esa rama sin explorarla.
- Si cumple $\Rightarrow$ se continúa construyendo hacia la siguiente posición.

Por ejemplo con $\{1, 2, 3\}$, si en la posición 0 se coloca el 3:

- Se prueba colocar 1 en posición 1: $3 \leq 2 \cdot 1 = 2$? **NO** $\rightarrow$ poda, no se exploran $[3, 1, 2]$ ni ninguna de sus variantes.
- Se prueba colocar 2 en posición 1: $3 \leq 2 \cdot 2 = 4$? **SÍ** $\rightarrow$ continúa explorando $[3, 2, \ldots]$.

La diferencia clave es que el backtracking nunca construye ni evalúa permutaciones que ya se sabe de antemano que serán inválidas, mientras que la fuerza bruta siempre las completa antes de descartarlas.

8.3. ¿Por qué `next_permutation` genera exactamente $n!$ permutaciones?

La función `next_permutation` recorre las permutaciones en orden lexicográfico estricto, una por una, sin repetir ni saltarse ninguna.

Al ordenar el arreglo de forma ascendente antes del bucle se garantizan dos condiciones:

1. Se comienza desde la permutación mínima (la más pequeña en orden lexicográfico), por ejemplo $[1, 2, 3]$.
2. Se termina en la permutación máxima (la más grande), por ejemplo $[3, 2, 1]$, momento en que `next_permutation` retorna `false` y el bucle termina.

Como entre la permutación mínima y la máxima de n elementos distintos existen exactamente $n!$ ordenamientos posibles, y `next_permutation` los recorre todos en orden sin repetir ninguno, se garantiza que genera exactamente $n!$ permutaciones.

Si el arreglo no estuviera ordenado inicialmente, el recorrido comenzaría desde la mitad y sólo generaría las permutaciones mayores al estado inicial, perdiéndose las anteriores y sin llegar a $n!$ en total.

9. Conclusiones

1. El algoritmo de fuerza bruta genera correctamente las $n!$ permutaciones y filtra las que cumplen la restricción $P[i] \leq 2 \cdot P[i+1]$, produciendo resultados exactos en todos los casos probados.

2. La complejidad temporal $O(n! \cdot n)$ hace que el algoritmo sea impracticable a partir de $n \approx 12$: el factorial crece mucho más rápido que cualquier función polinomial o exponencial de base fija, y para $n = 13$ se superan los 6×10^9 ciclos de trabajo.

3. La proporción de permutaciones válidas disminuye drásticamente con n: usando $\{1, 2, \ldots, n\}$, las válidas crecen como potencias de 2 mientras el total crece factorialmente. Para $n = 7$ la proporción ya es menor al 2 %, lo que evidencia la ineficiencia creciente de la fuerza bruta.

4. El número exacto de válidas depende tanto de n como de los valores específicos del conjunto: elementos muy separados entre sí reducen las válidas más drásticamente que elementos cercanos.

5. La función `next_permutation` garantiza exactamente $n!$ iteraciones siempre que el arreglo esté ordenado ascendentemente al inicio, condición necesaria para el correcto funcionamiento del algoritmo.

6. Un enfoque de backtracking podría mejorar significativamente la eficiencia al podar ramas inválidas durante la construcción de cada permutación, evitando explorar subárboles completos de soluciones que ya se sabe que serán inválidas.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[2] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012.

[3] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.

[4] Cppreference, *std::next_permutation*, 2024. Disponible en: https://en.cppreference.com/w/cpp/algorithm/next_permutation

[5] C. A. Álvarez Henao, *Material de clase — Análisis y Diseño de Algoritmos*. Universidad EAFIT, 2026.